Vibecoding 40 小時完整講義

- 目標：教會學生用 AI 工程化解決真實問題，並可按照講義做一次就有基本概念。

全課總覽（你在第一堂就要講清楚）

你會做出什麼（期末專案）

官網客服 Lead 系統（MVP），能做到：

1. 接收使用者訊息（簡版 chat endpoint）
2. 用 LLM 進行：
    - 意圖分類：報價/預約/售後/合作/應徵
    - 聯絡方式抽取：email/手機/LINE
    - 摘要：summary
3. 寫入 Google Sheet（MVP 必做）
4. 基本工程化：
    - 格式約束（JSON schema）
    - 錯誤處理
    - 去重/重試（idempotency）
    - 測試（最少 3–10 個）
    - secrets 不入版控
    - audit log（可追溯）

為什麼要這樣學

因為真實工作不是「寫出程式碼」而已，是：

- 需求能不能對齊？
- 東西能不能跑、能不能交接？
- 出錯怎麼辦？
- 可以追溯嗎？

這門課教的就是把 AI 變成「穩定產出」的一部分，而不是靠運氣。

課前準備（學生照著做）

方案 A（推薦）：Google Colab（零安裝）

優點：不怕環境炸裂、教室網路允許即可。

- 需要：Google 帳號
- 你要準備：一份 Colab notebook（我之後可幫你產生範本）

方案 B：本機 Python（工程感更強）

建議版本：Python 3.11+

1. 安裝 Python（macOS）

- 建議用 Homebrew：
  - 安裝 brew（若已有可跳過）
  - `brew install python`

2. 建立專案資料夾

```
mkdir vibecoding-leadbot
cd vibecoding-leadbot
```

3. 建立虛擬環境

```
python3 -m venv .venv
source .venv/bin/activate
python -V
```

4. 安裝套件（先安裝最小集合）

```
pip install fastapi uvicorn pydantic python-dotenv requests
```

5. 版本控制（強烈建議）

```
git init
```

專案固定結構（全班統一，避免混亂）

你要讓學生用同一套結構，講義才有用：

```
vibecoding-leadbot/
  README.md
  .env.example
  .gitignore
  app/
    __init__.py
    main.py
    settings.py
    schemas.py
    llm.py
    sheets.py
    audit.py
  tests/
```

```
    test_schemas.py
    test_extract.py
```

.gitignore（必做）

建立 `.gitignore`：

```
.venv/
__pycache__/
.env
.DS_Store
```

Part 1/5：第 1–4 堂（基礎與開局）

> 節奏（每堂 2 小時建議）：
>
> - 10m 回顧（看作業/看 repo）
> - 25m 教學（只講必要概念）
> - 60m 實作（跟做 + 巡迴）
> - 15m Review（看 diff、看 AC、看錯誤訊息）
> - 10m 作業說明（要可驗收）

第 1 堂：Vibecoding 入門（把需求寫成可交付）

本堂目標（學生要做出來的東西）

- 產出 1：`docs/spec.md`（或 README 內的一頁規格）
- 產出 2：第一版任務板（issues 或 checklist）

你要教的核心概念

1. **你不是要學生會寫所有程式**
    - 你要他們會「定義問題、定義驗收、驗證結果」
2. **AI 不是神，它是函式（function）**
    - 輸入不清楚 → 輸出就不可靠
3. **為什麼要寫規格（spec）**
    - 因為：人要對齊、AI 要對齊、測試要對齊

實作 1：建立 repo 與基本文件

1. 在專案根目錄建立 `README.md`

2. 建立 `docs/spec.md`

`docs/spec.md` 範本（學生照抄即可）：

```
# Lead 系統規格（v1）

## Problem
我們需要一個系統，能把官網客服對話轉成可跟進的 lead，避免線索流失。

## Users
- 客服/業務：要看到 lead、知道對方想要什麼、怎麼聯絡。
- 管理者：要能追溯（audit）。

## Scope（本期要做）
- 接收使用者訊息（簡版 API）
- 用 LLM 產生：intent / contact / summary
- 把 lead 寫入 Google Sheet

## Out of scope（本期不做）
- 不做自動回覆 LINE/FB
- 不做完整登入權限系統
- 不做排程推播

## Success Criteria
- 能跑起來、能 demo：輸入一段對話 → Sheet 出現一列 lead
- 能處理至少 5 種 intent：報價/預約/售後/合作/應徵
- 能抽取 email/手機/line（任一即可建立 lead）
```

實作 2：把需求變成「驗收標準（AC）」

你要教學生：**AC 是可驗證句子**，不是口號。

AC 範本（至少 8 條）：

```
## Acceptance Criteria（AC）
1. 呼叫 POST /chat，傳入 message 與 session_id，會回傳 JSON。
2. JSON 內包含：reply（客服回覆）與 extracted（抽取結果）。
3. extracted.intent 必屬於：報價/預約/售後/合作/應徵。
4. extracted 至少能抽到 1 種聯絡方式（email/phone/line）才算 lead。
5. 若聯絡方式都抽不到，系統不寫入 Sheet，並回傳提示（need_contact=true）。
6. 寫入 Sheet 成功時回傳 lead_id（或 row number）。
7. 重複送出同一 session_id + 相同 message，不會重複寫入（去重）。
8. 全流程有 audit log：記錄時間、session_id、intent、寫入狀態。
```

本堂建議示範 Prompt（老師示範用）

> 你要示範「怎麼讓 AI 產出規格與 AC」，而不是只示範寫碼。

Prompt（規格化）：

- 請 AI 產出 AC、Out of scope、風險。

```
你是產品經理 + 資深工程師。
請把以下想法整理成可交付的規格文件（markdown），並給出 8–12 條可驗證的 Acceptance Criteria。

想法：我要做一個官網客服 lead 系統，對話結束後抽取意圖（報價/預約/售後/合作/應徵）與聯絡方式（email/手機/line），寫到 Google Sheet。

限制：
- 本期不做登入權限
- 先用 API 方式 demo，不做完整前端
- 請提供 Out of scope 與風險清單
輸出：spec.md（markdown）
```

作業（可驗收）

1. `docs/spec.md` 完成（含 Scope/Out of scope/AC）
2. 任務板建立（至少 10 個 tasks，每個 task 要有驗收方式）

驗收方式（助教/講師怎麼看）

- Spec 是否能讓「沒聽課的人」也懂？
- AC 是否每條都能用 demo/測試驗證？

第 2 堂：任務切分（Tasks）與小步迭代（Small Diffs）

本堂目標

- 產出 1：任務板變成可執行（每個 task ≤10 分鐘）
- 產出 2：建立第一個最小可跑的服務（Hello API）

概念：什麼是好 task

好 task 需要：

- 有明確輸入/輸出
- 能在 10 分鐘內驗證
- 會修改哪些檔案

Bad task 例子：

- 「把系統做完」
- 「把 AI 串起來」

Good task 例子：

- 「建立 FastAPI 專案並提供 GET /health 回傳 ok」
- 「定義 Lead schema（Pydantic）並通過 3 個測試資料」

實作：建立 FastAPI Hello 服務

建立 `app/main.py`：

```
from fastapi import FastAPI

app = FastAPI(title="LeadBot")

@app.get("/health")
def health():
    return {"ok": True}
```

建立 `app/__init__.py`（空檔即可）。

啟動：

```
uvicorn app.main:app --reload --port 8000
```

測試：

- 打開瀏覽器：`http://127.0.0.1:8000/health`
- 看到 `{ "ok": true }`

作業

- 完成 /health
- README 寫清楚怎麼跑

第 3 堂：Prompt 生成（Generate）— 用 AI 產出專案骨架

本堂目標

- 產出 1：完整專案骨架（settings/schemas/llm/sheets/audit 檔案存在）
- 產出 2：能跑起來（/health 仍可用）

你要教的 Prompt 原則

- 指定「輸出格式」：檔案樹 + 每個檔案完整內容
- 指定「限制」：不要亂選套件、不要加過度複雜架構
- 指定「驗收」：我會跑 uvicorn，我會呼叫 /health

Prompt 範本（學生照貼）

```
你是一個資深 Python 工程師。
請為我生成一個最小可用的 FastAPI 專案骨架，用於官網客服 lead 系統。
```

```
需求：
- 需要有 app/main.py 提供 /health
- 需要有 app/settings.py 讀取環境變數（先只放 placeholder）
- 需要有 app/schemas.py 定義 LeadExtracted
schema（intent/contact/summary/confidence）
- 需要有 app/audit.py 提供 audit_log(event: dict) 先印出即可
- 需要有 app/llm.py 提供 extract_lead(message: str) 先回傳 mock 資料
- 需要有 app/sheets.py 提供 write_lead_to_sheet(data: dict) 先印出即可

限制：
- 先不要真的串任何 API key
- 不要引入資料庫
- 不要引入多餘框架

輸出：
1）檔案樹
2）每個檔案的完整內容
3）如何執行與測試
```

實作重點：先用 mock 讓流程跑通

這堂的關鍵是：

- LLM 先不用真的接
- Sheets 先不用真的寫
- 只要能走通資料流即可

作業

- 確保全專案能跑
- 提交一次 commit（訊息：chore: scaffold project）

第 4 堂：Prompt 修改（Edit）— 讓 AI 做「小改動」

本堂目標

- 產出 1：加入 POST /chat endpoint
- 產出 2：/chat 回傳固定 JSON（先不接真的 LLM）

先定義 /chat 的輸入輸出

Request

- session_id: string
- message: string

Response

- reply: string
- extracted: object（mock）

- need_contact: boolean

實作：建立 schemas

建立/補齊 `app/schemas.py`（範例）：

```
from pydantic import BaseModel, Field
from typing import Optional, Literal

Intent = Literal["報價", "預約", "售後", "合作", "應徵"]

class LeadExtracted(BaseModel):
    intent: Intent
    summary: str
    email: Optional[str] = None
    phone: Optional[str] = None
    line: Optional[str] = None
    confidence: float = Field(ge=0, le=1)

class ChatRequest(BaseModel):
    session_id: str
    message: str

class ChatResponse(BaseModel):
    reply: str
    extracted: LeadExtracted
    need_contact: bool
```

實作：建立 /chat endpoint（先用 mock）

在 `app/main.py` 加入：

```
from fastapi import FastAPI
from app.schemas import ChatRequest, ChatResponse, LeadExtracted

app = FastAPI(title="LeadBot")

@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    extracted = LeadExtracted(
        intent="報價",
        summary="使用者詢問報價，尚未留下聯絡方式。",
        email=None,
        phone=None,
        line=None,
        confidence=0.6,
    )
    need_contact = not (extracted.email or extracted.phone or
extracted.line)
    reply = "了解，我可以協助報價。方便留下您的 Email / 手機 / LINE 其中一項嗎？"
```

```
    return ChatResponse(reply=reply, extracted=extracted,
need_contact=need_contact)
```

測試（curl）：

```
curl -X POST http://127.0.0.1:8000/chat \
  -H 'Content-Type: application/json' \
  -d '{"session_id":"s1","message":"我想詢問一下你們的方案價格"}'
```

你應該看到 JSON 回傳（包含 reply/extracted/need_contact）。

本堂示範 Prompt（小改動）

```
請只修改 app/main.py，加入一個 POST /chat endpoint。
限制：不要更動 /health；不要新增新套件；回傳格式要符合 app/schemas.py 的
ChatResponse。
請輸出 unified diff。
```

作業

- /chat 可跑，回傳格式符合 schema
- README 加上 curl 測試範例

Part 1/5 完成檢查清單

- [ ] 有 docs/spec.md（含 Scope/Out of scope/AC）
- [ ] 有 /health
- [ ] 有 /chat（mock）
- [ ] 有 schemas（ChatRequest/ChatResponse/LeadExtracted）
- [ ] README 寫清楚如何啟動與測試
- [ ] 有至少 2 次 commit

下一步（Part 2/5 會做什麼）

Part 2/5 會進入真正核心：

- 真的接入 LLM（並強制輸出 JSON）
- 建立 schema 驗證 + 重試/fallback
- 串 Google Sheets（寫入、upsert、避免重複）

（你回我「繼續」我就會把 Part 2/5 直接補進同一份檔案。）

Part 2/5：第 5–8 堂（接入 LLM + 串 Google Sheets）

> 本 Part 的目標：把 Part 1 的 mock 變成「真的能抽取、真的能寫入」。你會完成：
>
> - 可控的 LLM 抽取（強制 JSON、驗證 schema、必要時重試/fallback）
> - Google Sheets 寫入（新增/更新、去重、欄位固定）
> - 最小 audit log（把每次處理記下來，方便稽核/除錯）

第 5 堂：Lead Schema 強化 + 驗證（讓資料可控）

本堂目標（要交什麼）

- 產出 1：`app/schemas.py` 完整化（包含 LeadWrite/LeadRow）
- 產出 2：`tests/test_schemas.py` 至少 5 個測試（包含成功與失敗）

你要先建立的核心概念

1. **LLM 輸出一定要被驗證**
   - 不驗證＝你把資料品質交給運氣
2. **Schema 是一種契約（contract）**
   - API / LLM / 落地（Sheets/DB）都要用同一份契約

Step 1：擴充 schemas

在 `app/schemas.py` 內新增一個「寫入用」資料結構（LeadWrite）。

> 設計原則：
>
> - extracted 是模型輸出
> - lead_write 是「準備落地」的資料（多了 metadata）

範例（可直接貼上，若你已有同名 class 請整合）：

```
from pydantic import BaseModel, Field
from typing import Optional, Literal
from datetime import datetime

Intent = Literal["報價", "預約", "售後", "合作", "應徵"]

class LeadExtracted(BaseModel):
    intent: Intent
    summary: str
    email: Optional[str] = None
    phone: Optional[str] = None
    line: Optional[str] = None
    confidence: float = Field(ge=0, le=1)

class LeadWrite(BaseModel):
    created_at: str
    session_id: str
    message: str
    intent: Intent
    confidence: float = Field(ge=0, le=1)
```

```
    summary: str
    email: Optional[str] = None
    phone: Optional[str] = None
    line: Optional[str] = None
    status: str = "new"

    @staticmethod
    def from_chat(session_id: str, message: str, extracted: LeadExtracted)
-> "LeadWrite":
        return LeadWrite(
            created_at=datetime.utcnow().isoformat(),
            session_id=session_id,
            message=message,
            intent=extracted.intent,
            confidence=extracted.confidence,
            summary=extracted.summary,
            email=extracted.email,
            phone=extracted.phone,
            line=extracted.line,
            status="new",
        )
```

Step 2：加入測試（先保證資料品質）

建立 `tests/test_schemas.py`：

```
import pytest
from app.schemas import LeadExtracted, LeadWrite


def test_extracted_ok():
    x = LeadExtracted(intent="報價", summary="詢問價格", email="a@b.com",
confidence=0.8)
    assert x.intent == "報價"


def test_extracted_confidence_range():
    with pytest.raises(Exception):
        LeadExtracted(intent="報價", summary="x", confidence=1.5)


def test_leadwrite_from_chat():
    extracted = LeadExtracted(intent="預約", summary="想預約",
phone="0912345678", confidence=0.9)
    lead = LeadWrite.from_chat("s1", "我想預約", extracted)
    assert lead.session_id == "s1"
    assert lead.intent == "預約"
    assert lead.phone == "0912345678"


def test_leadwrite_missing_contact_allowed_but_detectable():
```

```
    extracted = LeadExtracted(intent="合作", summary="談合作",
confidence=0.6)
    lead = LeadWrite.from_chat("s2", "想合作", extracted)
    assert (lead.email is None) and (lead.phone is None) and (lead.line is
None)


def test_status_default():
    extracted = LeadExtracted(intent="售後", summary="售後問題",
confidence=0.7)
    lead = LeadWrite.from_chat("s3", "售後", extracted)
    assert lead.status == "new"
```

安裝 pytest（如尚未）：

```
pip install pytest
pytest -q
```

本堂作業（可驗收）

- `pytest` 可以跑過
- commit 訊息：`test: add schema tests`

第 6 堂：LLM 抽取（強制 JSON、驗證、重試與 fallback）

> 我們做的是「合法、可控、可工程化」的 LLM 使用方式。

本堂目標

- 產出 1：`app/llm.py` 的 `extract_lead(message)` 變成真的會呼叫 LLM（或至少有可切換 mock/real）
- 產出 2：LLM 輸出一定會走過 schema 驗證
- 產出 3：失敗時有 fallback（不要整個服務掛掉）

重要設計：先定義 LLM 的輸入與輸出契約

我們把 LLM 的工作拆成兩步：

1. 產生回覆（reply）：自然語言
2. 抽取資料（extracted）：純 JSON

> 為了可控，本課程建議**先做抽取（extracted）**，回覆可以先用固定模板。

Step 1：設定檔與環境變數

建立 `.env.example`：

```
LLM_PROVIDER=mock
OPENAI_API_KEY=
OPENAI_MODEL=gpt-4.1-mini
OPENAI_API_BASE=https://api.openai.com/v1
```

建立 `app/settings.py`（簡化版）：

```
import os

class Settings:
    LLM_PROVIDER = os.getenv("LLM_PROVIDER", "mock")
    OPENAI_API_KEY = os.getenv("OPENAI_API_KEY")
    OPENAI_MODEL = os.getenv("OPENAI_MODEL", "gpt-4.1-mini")
    OPENAI_API_BASE = os.getenv("OPENAI_API_BASE",
"https://api.openai.com/v1")

settings = Settings()
```

> 注意：
>
> - `.env` 不要 commit
> - `.env.example` 要 commit

Step 2：LLM 抽取函式（可切換 mock/real）

建立/修改 `app/llm.py`：

```
import json
import requests
from app.settings import settings
from app.schemas import LeadExtracted


ALLOWED_INTENTS = ["報價", "預約", "售後", "合作", "應徵"]


def _mock_extract(message: str) -> LeadExtracted:
    # 你可以做很簡單的關鍵字判斷，確保 demo 可用
    intent = "報價"
    if "預約" in message:
        intent = "預約"
    elif "售後" in message or "壞" in message or "維修" in message:
        intent = "售後"
    elif "合作" in message:
        intent = "合作"
    elif "工作" in message or "應徵" in message:
        intent = "應徵"
```

```
    return LeadExtracted(
        intent=intent,
        summary=f"使用者詢問：{message[:30]}",
        email=None,
        phone=None,
        line=None,
        confidence=0.6,
    )


def _openai_extract(message: str) -> LeadExtracted:
    if not settings.OPENAI_API_KEY:
        raise RuntimeError("Missing OPENAI_API_KEY")

    prompt = (
        "你是客服助理，請從使用者訊息中抽取意圖與聯絡方式。\n"
        "請回傳『純 JSON』，不得輸出任何多餘文字。\n"
        "JSON schema：\n"
        "{\n"
        "  \"intent\": \"報價|預約|售後|合作|應徵\",\n"
        "  \"summary\": string,\n"
        "  \"email\": string|null,\n"
        "  \"phone\": string|null,\n"
        "  \"line\": string|null,\n"
        "  \"confidence\": number (0~1)\n"
        "}\n"
        f"使用者訊息：{message}\n"
    )

    req = {
        "model": settings.OPENAI_MODEL,
        "input": [
            {
                "role": "user",
                "content": [{"type": "input_text", "text": prompt}],
            }
        ],
        "response_format": {"type": "json_object"},
        "temperature": 0.2,
    }

    r = requests.post(
        f"{settings.OPENAI_API_BASE}/responses",
        headers={
            "Authorization": f"Bearer {settings.OPENAI_API_KEY}",
            "Content-Type": "application/json",
        },
        json=req,
        timeout=60,
    )
    r.raise_for_status()
    data = r.json()

    # 解析 Responses API 的 output_text
```

```
    text = None
    for item in data.get("output", []):
        for c in item.get("content", []):
            if c.get("type") == "output_text" and c.get("text"):
                text = c["text"]
                break
        if text:
            break
    if not text:
        raise RuntimeError("No output_text from Responses API")

    obj = json.loads(text)
    # schema 驗證
    return LeadExtracted(**obj)


def extract_lead(message: str) -> LeadExtracted:
    # 重試策略：只針對 LLM real 模式
    if settings.LLM_PROVIDER == "mock":
        return _mock_extract(message)

    try:
        return _openai_extract(message)
    except Exception:
        # fallback：至少要讓流程不掛
        return _mock_extract(message)
```

> 這裡的關鍵：
>
> - **永遠用 LeadExtracted(**obj) 驗證**
> - 失敗就 fallback（先求可用）
> - 真正工程版會把錯誤記到 audit log（下一堂補）

Step 3：把 /chat 接到 LLM

修改 `app/main.py`：

- 把 mock extracted 改成呼叫 `extract_lead(req.message)`
- 產生 lead_write（下一堂會寫 sheet）

範例片段：

```
from app.llm import extract_lead
from app.schemas import ChatRequest, ChatResponse

@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    extracted = extract_lead(req.message)
    need_contact = not (extracted.email or extracted.phone or
extracted.line)
    reply = "收到～我了解你的需求。方便留一下 Email / 手機 / LINE 其中一項嗎？"
```

```
    return ChatResponse(reply=reply, extracted=extracted,
need_contact=need_contact)
```

本堂作業

- 能切換 `LLM_PROVIDER=mock` 跑通
- 若有 OPENAI_API_KEY，可切到 `LLM_PROVIDER=openai`（或任意非 mock）看抽取結果
- commit：`feat: llm extraction with schema validation`

第 7 堂：audit log（讓每次處理可追溯）

本堂目標

- 產出 1：`app/audit.py` 真正寫入一個本機檔案（JSONL）
- 產出 2：每次 /chat 都會留下紀錄（含成功/失敗）

什麼是 JSONL（稽核最愛）

- 一行一個 JSON
- 方便 append
- 方便之後用 jq/grep/Excel 解析

Step 1：建立 audit.py

建立 `app/audit.py`：

```
import json
import os
from datetime import datetime

AUDIT_PATH = os.getenv("AUDIT_PATH", "audit.log.jsonl")


def audit_log(event: dict):
    record = {
        "ts": datetime.utcnow().isoformat(),
        **event,
    }
    with open(AUDIT_PATH, "a", encoding="utf-8") as f:
        f.write(json.dumps(record, ensure_ascii=False) + "\n")
```

Step 2：在 /chat 裡記錄

在 `app/main.py`：

- 先記錄 request
- 再記錄 extracted
- 若失敗記錄 error

範例（簡化版）：

```
from app.audit import audit_log

@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    audit_log({"event": "chat.request", "session_id": req.session_id})
    try:
        extracted = extract_lead(req.message)
        audit_log({
            "event": "chat.extracted",
            "session_id": req.session_id,
            "intent": extracted.intent,
            "confidence": extracted.confidence,
        })
    except Exception as e:
        audit_log({"event": "chat.error", "session_id": req.session_id,
"error": str(e)})
        raise

    need_contact = not (extracted.email or extracted.phone or
extracted.line)
    reply = "收到～方便留一下 Email / 手機 / LINE 其中一項嗎？"
    return ChatResponse(reply=reply, extracted=extracted,
need_contact=need_contact)
```

作業

- 跑 /chat 兩次
- 打開 `audit.log.jsonl` 確認有紀錄
- commit：`feat: audit log jsonl`

第 8 堂：串 Google Sheets（寫入、upsert、避免重複）

> 這堂是整個 MVP 最像工作用的部分。

本堂目標

- 產出 1：`app/sheets.py` 能把 LeadWrite 寫入 Google Sheet
- 產出 2：以 session_id 作為去重 key（簡單 upsert）
- 產出 3：/chat 在「有聯絡方式」時寫入 sheet

重要：Google Sheets 有兩條路

- 方式 1：**Apps Script Web App**（最適合工作坊/教學）
  - 優點：不用處理 service account、權限比較直覺
  - 缺點：要寫一點 Apps Script
- 方式 2：Google Sheets API + Service Account
  - 優點：工程化、企業常用

    - 缺點：設定比較多

本課程先用方式 1（容易成功）。方式 2 放到 Part 4/5 選配。

Step A：建立 Google Sheet 與 Apps Script

1. 建立一份 Google Sheet，命名：`LeadBot`
2. 建立表頭（第一列）：

- created_at
- session_id
- intent
- confidence
- summary
- email
- phone
- line
- message
- status

3. 進入 Extensions → Apps Script
4. 貼上以下程式（最小版）

```
function doPost(e) {
  const body = JSON.parse(e.postData.contents);
  const sheet =
SpreadsheetApp.getActiveSpreadsheet().getSheetByName('Sheet1');

  // 你可以改成你的工作表名稱
  // const sheet =
SpreadsheetApp.getActiveSpreadsheet().getSheetByName('Leads');

  // 簡單 upsert：找 session_id
  const sessionId = body.session_id;
  const values = sheet.getDataRange().getValues();
  let rowIndex = -1;

  // values[0] 是 header
  for (let i = 1; i < values.length; i++) {
    if (values[i][1] === sessionId) { // 第 2 欄是 session_id
      rowIndex = i + 1; // spreadsheet row is 1-indexed
      break;
    }
  }

  const row = [
    body.created_at,
    body.session_id,
    body.intent,
    body.confidence,
```

```
      body.summary,
      body.email || '',
      body.phone || '',
      body.line || '',
      body.message || '',
      body.status || 'new'
    ];

    if (rowIndex === -1) {
      sheet.appendRow(row);
    } else {
      sheet.getRange(rowIndex, 1, 1, row.length).setValues([row]);
    }

    return ContentService
      .createTextOutput(JSON.stringify({ ok: true, upserted: true }))
      .setMimeType(ContentService.MimeType.JSON);
  }
```

5. Deploy → New deployment → Web app

- Execute as：Me
- Who has access：Anyone with the link（教學用）
- 取得 URL（Web app URL）

> 這個 URL 就是你 Python 端要 POST 的 endpoint。

Step B：在 Python 端寫 sheets.py

在 `.env.example` 增加：

```
SHEETS_WEBHOOK_URL=
```

在 `app/settings.py` 增加：

```
SHEETS_WEBHOOK_URL = os.getenv("SHEETS_WEBHOOK_URL")
```

建立 `app/sheets.py`：

```
import requests
from app.settings import settings
from app.schemas import LeadWrite


def write_lead_to_sheet(lead: LeadWrite) -> dict:
    if not settings.SHEETS_WEBHOOK_URL:
```

```
        raise RuntimeError("Missing SHEETS_WEBHOOK_URL")

    r = requests.post(
        settings.SHEETS_WEBHOOK_URL,
        json=lead.model_dump(),
        timeout=30,
    )
    r.raise_for_status()
    return r.json()
```

Step C：在 /chat 裡「有聯絡方式才寫」

在 `app/main.py`：

- extracted -> lead_write
- need_contact = True 就不寫
- need_contact = False 才寫入

範例（概念版）：

```
from app.schemas import LeadWrite
from app.sheets import write_lead_to_sheet

@app.post("/chat", response_model=ChatResponse)
def chat(req: ChatRequest):
    extracted = extract_lead(req.message)
    need_contact = not (extracted.email or extracted.phone or
extracted.line)

    if not need_contact:
        lead = LeadWrite.from_chat(req.session_id, req.message, extracted)
        res = write_lead_to_sheet(lead)
        audit_log({"event": "sheet.write", "session_id": req.session_id,
"res": res})

    reply = "收到～" + ("方便留聯絡方式嗎？" if need_contact else "已幫你記下來，
稍後會有人跟你聯絡。")
    return ChatResponse(reply=reply, extracted=extracted,
need_contact=need_contact)
```

本堂驗收（課堂上就要驗）

1. 用 curl 打 /chat，內容包含 email 或 phone 或 line（你可以直接在 message 裡給）
2. Sheet 會新增或更新一列
3. audit.log.jsonl 會出現 sheet.write

本堂作業

- 完成 Sheets 寫入
- README 加上：
  - 如何建立 Apps Script
  - 如何設定 SHEETS_WEBHOOK_URL
  - 如何測試
- commit：`feat: write leads to google sheets`

Part 2/5 完成檢查清單

- [ ] LLM 抽取有 schema 驗證（不管用 mock/real）
- [ ] audit log 有記錄 request/抽取/寫入
- [ ] Google Sheets 能新增/更新（session_id 去重）
- [/chat 在有聯絡方式時會寫入 Sheet

下一步（Part 3/5）

Part 3/5 會進入「可靠性」：

- 錯誤處理標準化（error_code）
- idempotency 與去重策略強化
- 測試：抽取/寫入/錯誤的最小回歸
- 對 LLM 的重試策略（JSON 不合法、欄位缺失）

Part 3/5：第 9–12 堂（可靠性：錯誤處理、去重、重試、測試）

> 本 Part 的目標：讓你的系統「不會因為一個小錯誤就整個掛掉」，而且可以放心交接。

第 9 堂：錯誤處理標準化（工程化必修）

本堂目標

- 產出 1：統一錯誤回應格式（error_code / message / detail）
- 產出 2：/chat 遇到常見錯誤時，不會 500

概念：為什麼要統一錯誤格式

- 前端/呼叫端才知道怎麼處理
- 稽核/監控才好做
- AI 抽取失敗、Sheets 寫入失敗，都要能被分類

Step 1：定義錯誤代碼表（可放 docs/errors.md）

建立 `docs/errors.md`：

```
# Error Codes

- LLM_JSON_INVALID：LLM 回傳不是合法 JSON
- LLM_SCHEMA_INVALID：LLM JSON 欄位缺失或型別錯
- SHEETS_WRITE_FAILED：寫入 Google Sheet 失敗
- MISSING_SHEETS_WEBHOOK：未設定 SHEETS_WEBHOOK_URL
- INTERNAL_ERROR：未知錯誤
```

Step 2：建立自訂 Exception

建立 `app/errors.py`：

```
class AppError(Exception):
    def __init__(self, code: str, message: str, detail: str | None =
None):
        super().__init__(message)
        self.code = code
        self.message = message
        self.detail = detail
```

Step 3：在 FastAPI 加入 exception handler

在 `app/main.py` 加入：

```
from fastapi import Request
from fastapi.responses import JSONResponse
from app.errors import AppError

@app.exception_handler(AppError)
def app_error_handler(request: Request, exc: AppError):
    return JSONResponse(
        status_code=400,
        content={"ok": False, "error_code": exc.code, "message":
exc.message, "detail": exc.detail},
    )
```

Step 4：把 LLM/Sheets 的錯誤轉成 AppError

- 在 `app/llm.py` 或呼叫點 catch 後 raise AppError
- 在 `app/sheets.py` catch requests error 後 raise AppError

範例（sheets.py）：

```
import requests
from app.errors import AppError
```

```
def write_lead_to_sheet(lead):
    if not settings.SHEETS_WEBHOOK_URL:
        raise AppError("MISSING_SHEETS_WEBHOOK", "未設定 Google Sheets
webhook")

    try:
        r = requests.post(settings.SHEETS_WEBHOOK_URL,
json=lead.model_dump(), timeout=30)
        r.raise_for_status()
        return r.json()
    except Exception as e:
        raise AppError("SHEETS_WRITE_FAILED", "寫入 Google Sheet 失敗",
str(e))
```

本堂作業

- 故意把 SHEETS_WEBHOOK_URL 清空，/chat 會回傳 ok:false + error_code
- commit：`feat: standardized error responses`

第 10 堂：idempotency（去重策略）

本堂目標

- 產出 1：定義 idempotency key
- 產出 2：重複請求不會重複寫入

推薦策略（MVP 夠用）

- key = session_id + message 的 hash（或 request_id）
- 先記錄已處理的 key（MVP 用本機檔案/記憶體；工程版用 DB）

Step 1：建立 idempotency 模組

建立 `app/idempotency.py`（最小版，使用本機 JSON 檔）：

```
import json
import os

PATH = os.getenv("IDEMPOTENCY_STORE", "idempotency.json")


def _load():
    if not os.path.exists(PATH):
        return {}
    with open(PATH, "r", encoding="utf-8") as f:
        return json.load(f)


def _save(data: dict):
```

```
    with open(PATH, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)


def seen(key: str) -> bool:
    data = _load()
    return key in data


def mark(key: str, value: dict):
    data = _load()
    data[key] = value
    _save(data)
```

Step 2：在 /chat 寫入前先檢查

- key 建議用 `f"{session_id}:{message}"` 再做 sha256

`app/utils.py`：

```
import hashlib

def sha256_text(text: str) -> str:
    return hashlib.sha256(text.encode("utf-8")).hexdigest()
```

在 `app/main.py`：

```
from app.idempotency import seen, mark
from app.utils import sha256_text

key = sha256_text(req.session_id + ":" + req.message)
if seen(key):
    audit_log({"event": "idempotency.hit", "key": key, "session_id":
req.session_id})
    # 你可以直接回覆成功，但不要再寫入
else:
    # 寫入 sheet 成功後 mark
    mark(key, {"session_id": req.session_id})
```

作業

- 同一個請求打兩次，Sheet 不會多一列
- commit：`feat: idempotency store`

第 11 堂：LLM 重試策略（JSON 不合法/欄位缺失）

本堂目標

- 產出：LLM 抽取至少有「1 次重試」

概念：LLM 失敗的兩種常見型態

1. 回傳不是 JSON（夾雜文字）
2. JSON 但欄位不對（schema invalid）

最小重試策略

- 第一次：正常 prompt
- 第二次：補一段「你剛剛輸出格式不正確，請只輸出 JSON」

> 你不需要寫很複雜，重點是「有策略、可觀測」。

示意（llm.py）：

```
def extract_with_retry(message: str) -> LeadExtracted:
    try:
        return _openai_extract(message)
    except Exception as e1:
        # 第二次用更嚴格提示
        return _openai_extract("請只輸出純 JSON，不要任何多餘文字。使用者訊息：" +
message)
```

作業

- 故意讓 prompt 變鬆，觀察重試是否生效（看 audit log）
- commit：`feat: llm retry strategy`

第 12 堂：測試（抽取/寫入/錯誤的最小回歸）

本堂目標

- 產出：tests 至少 10 個（含成功與失敗）

測試策略（從最便宜開始）

1. schema 測試（已做）
2. 抽取測試：mock provider 下的 intent 判斷
3. sheets 測試：用 monkeypatch 把 requests.post 替換掉
4. /chat endpoint 測試：fastapi TestClient

Step 1：安裝測試工具

```
pip install pytest httpx
```

Step 2：建立 API 測試

建立 `tests/test_chat_api.py`：

```
from fastapi.testclient import TestClient
from app.main import app

client = TestClient(app)


def test_health():
    r = client.get("/health")
    assert r.status_code == 200
    assert r.json()["ok"] is True


def test_chat_returns_schema():
    r = client.post("/chat", json={"session_id": "s1", "message": "我想詢問
報價"})
    assert r.status_code == 200
    data = r.json()
    assert "reply" in data
    assert "extracted" in data
    assert "need_contact" in data
```

Step 3：建立 sheets 單元測試（monkeypatch）

建立 `tests/test_sheets.py`：

```
from app.schemas import LeadExtracted, LeadWrite
from app.sheets import write_lead_to_sheet

class DummyResp:
    def raise_for_status(self):
        return None
    def json(self):
        return {"ok": True}


def test_write_lead_to_sheet(monkeypatch):
    def fake_post(url, json, timeout):
        return DummyResp()

    monkeypatch.setenv("SHEETS_WEBHOOK_URL", "https://example.com")

    import app.settings
    app.settings.settings.SHEETS_WEBHOOK_URL = "https://example.com"

    extracted = LeadExtracted(intent="報價", summary="x", confidence=0.7,
email="a@b.com")
    lead = LeadWrite.from_chat("s1", "msg", extracted)
```

```
    monkeypatch.setattr("requests.post", fake_post)
    res = write_lead_to_sheet(lead)
    assert res["ok"] is True
```

作業

- `pytest -q` 全過
- commit：`test: add api and sheets tests`

Part 3/5 完成檢查清單

- [ ] 錯誤回應格式統一（error_code）
- [ ] idempotency 去重有效
- [ ] LLM 有至少 1 次重試策略
- [ ] tests >= 10 且可跑

下一步（Part 4/5）

Part 4/5 會把系統「工程化到可上線」：

- Docker 化
- 環境分離（dev/staging/prod）
- 健康檢查、回滾策略
- 安全：secrets、權限、PII 遮罩
- audit log 強化（查詢/聚合）

Part 4/5：第 13–16 堂（工程化：Docker、部署、安全、稽核）

> 本 Part 的目標：讓你的專案不只是在你電腦能跑，而是「別人的電腦也能跑」、「部署也可控」。

第 13 堂：Docker 化（可重複啟動）

本堂目標

- 產出：Dockerfile + 可啟動服務

Step 1：requirements.txt

在專案根目錄建立 `requirements.txt`：

```
fastapi
uvicorn
pydantic
python-dotenv
```

```
requests
pytest
httpx
```

Step 2：Dockerfile

建立 `Dockerfile`：

```
FROM python:3.11-slim

WORKDIR /app

COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 8000
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

Step 3：docker-compose.yml

建立 `docker-compose.yml`：

```
services:
  leadbot:
    build: .
    ports:
      - "8000:8000"
    env_file:
      - .env
```

啟動：

```
docker compose up --build
```

驗證：

- `http://127.0.0.1:8000/health`

作業

- Docker 可啟動
- commit：`chore: dockerize service`

第 14 堂：環境分離（dev/staging/prod）+ 設定治理

本堂目標

- 產出：設定結構清楚，避免「上線用錯 key」

概念

- `.env.dev`、`.env.staging`、`.env.prod`（或用部署平台管理）
- 程式碼只讀環境變數，不寫死

Step 1：settings.py 強化

把 settings.py 改成集中管理（至少把所有 key 列出來）：

- LLM_PROVIDER
- OPENAI_API_KEY / MODEL / BASE
- SHEETS_WEBHOOK_URL
- AUDIT_PATH
- IDEMPOTENCY_STORE

Step 2：.env.example 完整化

```
LLM_PROVIDER=mock
OPENAI_API_KEY=
OPENAI_MODEL=gpt-4.1-mini
OPENAI_API_BASE=https://api.openai.com/v1
SHEETS_WEBHOOK_URL=
AUDIT_PATH=audit.log.jsonl
IDEMPOTENCY_STORE=idempotency.json
```

作業

- README 新增「設定說明」章節
- commit：`docs: add env configuration`

第 15 堂：安全基礎（secrets、權限、PII）

本堂目標

- 產出：Security checklist + PII 遮罩策略

你要教學生的 5 個安全底線

1. secrets 不進 repo（.env 不 commit）
2. 回傳給前端的資料要最小化（不要把全部 audit 原文都回傳）
3. PII（email/phone/line）要遮罩展示
4. log 不要記錄完整敏感資訊（可記 hash 或末四碼）

5. 依資料分級決定是否允許用 LLM（公司政策）

Step 1：PII 遮罩工具

建立 `app/pii.py`：

```
import re

def mask_email(email: str) -> str:
    if not email or "@" not in email:
        return email
    name, domain = email.split("@", 1)
    if len(name) <= 2:
        return "*" * len(name) + "@" + domain
    return name[0] + "***" + name[-1] + "@" + domain


def mask_phone(phone: str) -> str:
    if not phone:
        return phone
    # 只留末 3-4 碼
    digits = re.sub(r"\D", "", phone)
    if len(digits) <= 4:
        return "*" * len(digits)
    return "*" * (len(digits) - 4) + digits[-4:]
```

Step 2：建立 security checklist（docs/security.md）

```
# Security Checklist

- [ ] .env 不在 repo
- [ ] API key 只存在部署環境
- [ ] 日誌不記錄明文密碼/金鑰
- [ ] PII 顯示需遮罩
- [ ] 外部 webhook URL 不公開（正式版）
- [ ] 依資料分級決定能否傳給 LLM
```

作業

- 在回傳或展示時套用遮罩（至少在後台顯示）
- commit：`feat: pii masking + security checklist`

第 16 堂：稽核（audit）強化：查詢、聚合、追溯

本堂目標

- 產出：最簡 audit 查詢方式（CLI 或 endpoint）

Step 1：新增 /audit/search（簡化）

在 `app/main.py` 加一個簡單 endpoint：

- 以 session_id 查 audit
- 回傳最近 N 筆

（示意，實務上要做權限控管；本課程先示範概念）

Step 2：audit 檔案輪替（選配）

- 每天一檔：audit-YYYY-MM-DD.jsonl
- 防止單檔爆大

作業

- 能查到某個 session 的處理紀錄
- commit：`feat: audit search endpoint`

Part 4/5 完成檢查清單

- [ ] Docker 可啟動
- [ ] 設定集中管理、.env.example 完整
- [ ] 有 PII 遮罩與 security checklist
- [ ] audit 可查詢/追溯

下一步（Part 5/5）

Part 5/5 是收斂與上線：

- Demo script（5 分鐘）
- README 完整化（安裝/設定/測試/常見問題）
- 回歸測試一鍵跑
- 最終驗收（依 AC 當場驗）

Part 5/5：第 17–20 堂（交付收斂：文件、Demo、驗收、上線清單）

> 本 Part 的目標：把作品做成「交出去別人也能用」的狀態。

第 17 堂：README 寫到能交接（不是裝飾）

本堂目標

- 產出：README 至少包含以下章節

README 建議結構（學生照抄）：

```
# LeadBot

## 1. 這是什麼？
（用 3 句話講清楚：解決什麼問題、誰用、產出什麼）

## 2. 功能
- /health
- /chat
- 寫入 Google Sheets
- audit log

## 3. 快速開始（本機）
```bash
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
cp .env.example .env
uvicorn app.main:app --reload --port 8000
```
```

4. 設定（環境變數）

- LLM_PROVIDER
- OPENAI_API_KEY
- SHEETS_WEBHOOK_URL
- AUDIT_PATH

5. 如何測試

```
pytest -q
```

6. Demo（curl 範例）

```
curl -X POST http://127.0.0.1:8000/chat ...
```

7. 常見問題

- Q：為什麼寫不進 Sheet？
- Q：LLM 回傳不是 JSON 怎麼辦？

8. Roadmap

（下一步想做什麼）

```
### 作業
- README 寫到「助教照著跑能成功」
- commit：`docs: improve README for handoff`

---

## 第 18 堂：Demo script（5 分鐘，像跟老闆報告）

### 本堂目標
- 產出：`docs/demo-script.md`

Demo script 範本：
```md
# Demo Script（5 分鐘）

## 0. 問題（30 秒）
官網客服對話容易流失線索，我們需要把對話轉成可跟進的 lead。

## 1. 我們做了什麼（30 秒）
使用者訊息 → LLM 抽取 intent/contact/summary → 寫入 Google Sheets → 可追溯
audit。

## 2. 現場 Demo（3 分鐘）
1）打 /health
2）打 /chat：一段報價詢問（含 email）
3）打開 Google Sheet：看到新增/更新
4）打開 audit log：看到這次處理紀錄

## 3. 可靠性與安全（1 分鐘）
- 有 schema 驗證 + retry + fallback
- 有 idempotency 去重
- secrets 不入 repo
- audit 可追溯

## 4. 下一步（30 秒）
- 加上後台 UI
- 權限/登入
- DB 正式化
```

作業

- demo script 完成
- commit：`docs: add demo script`

第 19 堂：最終回歸（一鍵檢查清單）

本堂目標

- 產出：`make check` 或 `./scripts/check.sh`
```

建立 `scripts/check.sh`：

```
#!/usr/bin/env bash
set -e

echo "[1/4] Running tests..."
pytest -q

echo "[2/4] Starting server (smoke test)..."
# 這裡簡化，你也可以只做 /health

echo "[3/4] Checking formatting (optional)"

echo "[4/4] Done"
```

賦權：

```
chmod +x scripts/check.sh
```

作業

- check script 可跑
- commit：`chore: add check script`

第 20 堂：稽核式驗收（照 AC 一條條驗）

本堂驗收流程（講師照這份做）

1. 先看 repo：
    - 是否有 README
    - 是否有 .env.example
    - 是否有 tests
2. 依 AC 驗收（範例）：
    - 打 /chat 回傳符合 schema
    - intent 在允許集合內
    - need_contact 行為正確
    - 有聯絡方式才寫入
    - 重複請求不重複寫入
    - audit log 有紀錄
3. 讓別組照 README 跑一次（最硬但最有效）

期末交付包

- Git repo
- docs/spec.md
- docs/demo-script.md

- audit log sample（可選）
- Google Sheet 欄位說明

Part 5/5 完成檢查清單（畢業條件）

- ☐ 可 demo：輸入 → 抽取 → 寫入 Sheet
- ☐ schema 驗證 + retry/fallback
- ☐ 統一錯誤格式
- ☐ idempotency 去重
- ☐ tests 可跑（>=10）
- ☐ Docker 可啟動（加分）
- ☐ README + demo-script 完整

全課附錄：常見問題（FAQ）與排錯清單

FAQ 1：為什麼 LLM 回傳不是 JSON？

- 原因：prompt 不夠嚴格、模型在「解釋」
- 解法：
    1. 使用 response_format（如果支援）
    2. 加 retry：要求只輸出 JSON
    3. schema 驗證失敗就重試

FAQ 2：Sheets 寫不進去

- 檢查：
    - SHEETS_WEBHOOK_URL 是否正確
    - Apps Script 是否部署為 Web app
    - 權限是否允許（Anyone with link）

FAQ 3：重複寫入

- 檢查：
    - session_id 是否固定
    - idempotency key 是否正確
    - Apps Script 是否真的做 upsert

FAQ 4：部署後環境變數沒讀到

- 檢查：
    - docker-compose 是否有 env_file
    - 平台是否有設定 secret

最後提醒

Vibecoding 的關鍵不是「AI 能寫多少」，而是：

- 你能不能把問題寫清楚（spec）
- 你能不能定義可驗收標準（AC）
- 你能不能用小步迭代把它推到可用
- 你能不能留下一套可交付、可維運的東西

做到這些，你才是真的在用 AI 放大自己。